IRWA Final Project - Part 2

Indexing and Evaluation

Group members: Andreu Clotas Burgas, u232071 · Elena Andreev Apukhtina, u231981 · Carolina Henao Fernandez, u232758

Repository: github.com/elenagotlost/irwa-search-engine

tag/commit: IRWA-2026-part-2

Part 2 builds the search engine on top of the corpus processed in Part 1 (data/processed_corpus.pkl, 28,080 products). The indexing, AND retrieval and TF-IDF ranking live in `myapp/core/algorithms.py` and the evaluation metrics in `myapp/search/evaluation.py`, so Parts 3-4 reuse them. Queries go through the same `preprocess_text` function as the documents, otherwise query terms and index terms would not match.

The notebook `project_progress/part_2/part_2.ipynb` runs everything and produces every number and figure in this report. The metric tables are also saved as `results_validation.csv` and `results_test_queries.csv`.

1. Indexing

1.1 Inverted index

Following the Part 1 decision, the index is built on **all_tokens** (title + description + informative product_details values + brand). The per-field token columns are kept for field weighting in Part 3. create_index makes one pass over the corpus and returns three structures:

- **index[term]** = [pid_1, pid_2, …]: the postings list, the vocabulary structure seen in the labs.
- **tf[term][pid]** = count / ‖d‖: term frequency normalised by the Euclidean length of the document vector, so products with long descriptions do not win just for being long.
- **idf[term]** = log(N / df), with N = 28,080.

Building the index takes about 0.2 s.

Statistic	Value
Vocabulary (distinct terms)	5,710
Total postings	795,585
Postings list length: median / mean	6 / 139.3
Longest postings list ("wear")	20,784 products
Terms that appear in a single product	1,342 (23 %)

Table 1. Size of the inverted index.

As Heaps' law predicted in Part 1, the vocabulary is small but the distribution is very skewed: half of the terms appear in 6 products or fewer, while a few template words (wear, cotton, western, sleeve, neck, women, men) appear in more than 13,000

products each. These long lists dominate the cost of an AND query, so the intersection should start from the shortest list (Section 2.5).

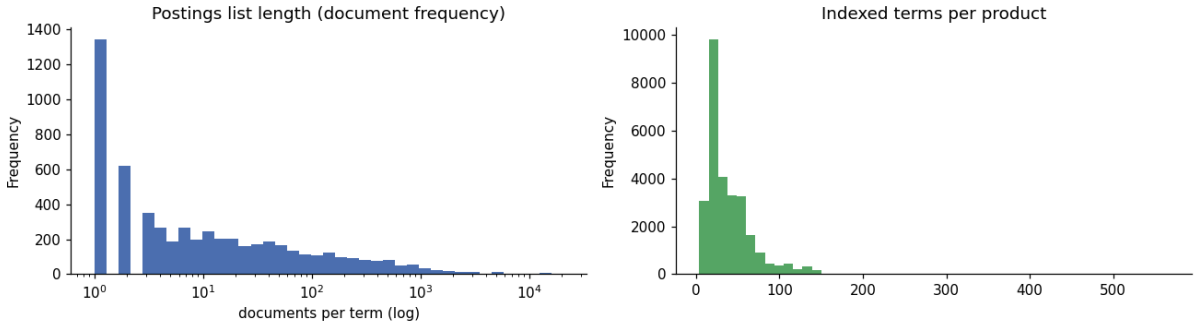


Figure 1. Postings list length (document frequency, log scale) and number of indexed terms per product.

1.2 Conjunctive (AND) queries

conjunctive_search intersects the postings lists of all the query terms, so a product is returned only if it contains **every** term. If one term is not in the vocabulary, the result is empty.

Query	Processed terms (df)	AND matches
women full sleeve sweatshirt cotton	women (13,702), full (7,174), sleeve (17,911), sweatshirt (1,418), cotton (19,866)	479
men slim jeans blue	men (13,410), slim (5,666), jean (2,436), blue (7,104)	218

Table 2. Validation queries after pre-processing and number of products that contain all their terms.

1.3 Test queries

To choose our queries we ranked the terms by document frequency and by TF-IDF mass (the sum of tf · idf over the products that contain the term). The most popular terms are the template dimensions of the titles: gender (men, women), product type (tshirt, shirt, sweatshirt), colour (black, blue, white) and fit/style (slim, polo, full). Very frequent terms alone (wear, cotton) are not useful queries because almost every product contains them, so each test query combines a **gender**, a **product type** and one or two **attributes**, which is how users usually search a fashion catalogue. We also covered different parts of the catalogue: topwear, ethnic wear, winter wear, footwear and outerwear.

#	Query	Structure	Terms (df)	AND matches
1	men cotton polo tshirt	gender + fabric + style + product	men (13,410), cotton (19,866), polo (3,040), tshirt (13,037)	1,213
2	women cotton kurta	gender + fabric + product (ethnic wear)	women (13,702), cotton (19,866), kurta (785)	358
3	men hooded sweatshirt	gender + style + product	men (13,410), hood (1,031), sweatshirt (1,418)	323
4	men running shoes	gender + use + product (footwear)	men (13,410), run (772), shoe (526)	37
5	women blue denim jacket	gender + colour + fabric + product	women (13,702), blue (7,104), denim (1,766), jacket (1,093)	27

Table 3. Our five test queries.

The queries go from very broad (1,213 matches) to very narrow (27 matches), which lets us see how the system behaves when the AND filter leaves many or few candidates.

1.4 TF-IDF ranking

rank_documents scores every product returned by the AND filter with the dot product of the query and document TF-IDF vectors:

- document weight: w(t,d) = tf(t,d) / ‖d‖ · idf(t)
- query weight: w(t,q) = tf(t,q) / ‖q‖ · idf(t)
- score(q,d) = Σ over t in q of w(t,q) · w(t,d)

Results are sorted by score and ties are broken by pid, so the ranking is always the same. search_in_corpus chains the three steps (pre-process the query, AND filter, rank) and returns the top-k (pid, score) pairs. For both validation queries the top 10 results are exactly the expected products ("Full Sleeve Solid Women Sweatshirt", "Slim Men Blue Jeans"), with scores between 2.0 and 3.0.

2. Evaluation

2.1 Implementation of the metrics

The metrics follow the format used in the labs: for each query, y_true holds the relevance label of every judged product and y_score the score our system gives it. top_k_labels sorts the judged products by score and keeps the first K. Judged products that the AND search does not return get score 0 and never count as retrieved. R is the number of judged relevant products of the query, and rel_i is the label of the product at rank i.

Metric	Definition
P@K	(1/K) · Σ rel_i, i = 1..K
R@K	(1/R) · Σ rel_i, i = 1..K
AP@K	mean of P@i over the ranks i ≤ K that hold a relevant product
F1@K	2 · P@K · R@K / (P@K + R@K)
MAP@K	mean of AP@K over the queries
MRR@K	mean over the queries of 1 / rank of the first relevant product in the top K (0 if none)
NDCG@K	DCG@K / IDCG@K, with DCG@K = Σ rel_i / log2(i + 1) and IDCG the DCG of the ideal ranking

Table 4. Implemented metrics (myapp/search/evaluation.py).

We checked every function against values computed by hand on a toy query (six judged products, three relevant, the last one not returned by the search) and checked that the mean of AP@K and RR@K over the queries equals map_at_k and mrr_at_k.

2.2 Results on validation_labels.csv

Metric	Q1 K=5	Q1 K=10	Q1 K=20	Q2 K=5	Q2 K=10	Q2 K=20
P@K	1.000	1.000	0.650	0.800	0.600	0.300
R@K	0.385	0.769	1.000	0.400	0.600	0.600
AP@K	1.000	1.000	0.980	1.000	0.948	0.948
F1@K	0.556	0.870	0.788	0.533	0.600	0.400
NDCG@K	1.000	1.000	0.995	0.869	0.716	0.716

Table 5. Query 1: women full sleeve sweatshirt cotton. Query 2: men slim jeans blue.

Metric	K=5	K=10	K=20
MAP@K	1.000	0.974	0.964
MRR@K	1.000	1.000	1.000

Table 6. Metrics over the two validation queries.

2.3 Ground truth for our test queries

We acted as expert judges. For each of our five queries we took the top-20 products returned by the system and the three of us assigned a binary label: **1** if the product is what a user typing the query wants, **0** otherwise. We looked at the title, the description and the product details (fabric, ideal for, hooded, type) of each product. The criteria were:

- **Q1 men cotton polo tshirt:** a men's polo-neck T-shirt in cotton (pure cotton or a cotton blend).
- **Q2 women cotton kurta:** a women's kurta in cotton or cotton blend; kurta sets (kurta + pyjama / churidar) count as relevant.
- **Q3 men hooded sweatshirt:** a men's sweatshirt with a hood. The titles never say "hooded"; the hood is in the details, which is why we index them.
- **Q4 men running shoes:** men's running or sports shoes. Socks, loafers, sneakers, lace-ups and party-wear shoes are not relevant.
- **Q5 women blue denim jacket:** a women's denim (or denim-blend) jacket. Shirts, T-shirts and vests are not relevant.

The labels are in project_progress/part_2/test_queries_to_label.csv (query_id, query, rank, pid, title, fabric, color, label). Since only the top 20 were judged (pooling), R is the number of relevant products **within the judged pool**, and the evaluation uses y_score = 1/rank so the metrics follow the order in which the products were returned. We checked that the current engine still returns exactly the same 20 products in the same order.

#	Query	Judged	Relevant	Non-relevant products (rank)
1	men cotton polo tshirt	20	19	track pants (20)
2	women cotton kurta	20	20	-
3	men hooded sweatshirt	20	20	-
4	men running shoes	20	11	socks (4); loafers, sneakers, lace-ups, party wear (13-20)
5	women blue denim jacket	20	13	shirt, T-shirts, vest (14-20)

Table 7. Relevance judgments of the test queries.

Metric	Q1	Q2	Q3	Q4	Q5	Mean
P@5	1.000	1.000	1.000	0.800	1.000	0.960
R@5	0.263	0.250	0.250	0.364	0.385	0.302
F1@5	0.417	0.400	0.400	0.500	0.556	0.454
AP@5	1.000	1.000	1.000	0.950	1.000	0.990
RR@5	1.000	1.000	1.000	1.000	1.000	1.000
NDCG@5	1.000	1.000	1.000	0.854	1.000	0.971
P@10	1.000	1.000	1.000	0.900	1.000	0.980
R@10	0.526	0.500	0.500	0.818	0.769	0.623
F1@10	0.690	0.667	0.667	0.857	0.870	0.750
AP@10	1.000	1.000	1.000	0.906	1.000	0.981
RR@10	1.000	1.000	1.000	1.000	1.000	1.000
NDCG@10	1.000	1.000	1.000	0.905	1.000	0.981
P@20	0.950	1.000	1.000	0.550	0.650	0.830
R@20	1.000	1.000	1.000	1.000	1.000	1.000
F1@20	0.974	1.000	1.000	0.710	0.788	0.894
AP@20	1.000	1.000	1.000	0.907	1.000	0.981
RR@20	1.000	1.000	1.000	1.000	1.000	1.000
NDCG@20	1.000	1.000	1.000	0.967	1.000	0.993

Table 8. Metrics of the test queries. In the Mean column, AP@K is MAP@K and RR@K is MRR@K.

2.4 Comparison of the metrics and analysis of the results

What each metric tells us.

- **P@K** is the share of the first K results that are relevant. It is easy to read and is what a user sees on the first page, but it ignores the order inside the top K and how many relevant products exist. Its maximum is R/K when R < K, so P@20 for validation query 2 (R = 10) can never be above 0.5.
- **R@K** is the share of all the relevant products that are in the top K. It measures coverage and is the only metric here that shows products the system never retrieves. Its maximum is K/R, so R@5 is low whenever there are many relevant products (0.25 for our queries 2 and 3 even with a perfect ranking).
- **F1@K** combines precision and recall with a harmonic mean, so it is high only when both are. With our numbers it mainly follows recall at K = 5 and precision at K = 20.
- **AP@K** looks at the order: it averages the precision at every rank where a relevant product appears, so a relevant product at rank 1 counts more than one at rank 10. In our implementation (the lab version) it is divided by the number of relevant products found in the top K, not by R, so it does not penalise relevant products that are never retrieved. **MAP** is its mean over the queries and is the usual single number to compare systems.
- **RR@K** only looks at the rank of the first relevant product; **MRR** is its mean. It matters for navigational searches where the user wants one good result, and says nothing about the rest of the list.

- **NDCG@K** also rewards relevant products at the top (logarithmic discount) and is normalised by the ideal ranking of all the judged relevant products. Unlike our AP, it does penalise missing relevant products, and it is the only metric that can use graded labels (2 = perfect, 1 = partly relevant).

Validation queries. Query 1 is almost perfect: the first 10 results are relevant, the 13 relevant products are all retrieved within the top 20 (R@20 = 1) and only one non-relevant product is ranked above a relevant one (AP@20 = 0.980, NDCG@20 = 0.995). Query 2 shows the main weakness of the system: **4 of its 10 relevant products are not returned at all** by the AND filter, because they do not contain all four terms in the indexed text. Recall stops at 0.6, P@20 is 0.3 and NDCG@20 drops to 0.716. Its AP@20 is still 0.948 because the 6 relevant products that are found are ranked near the top, which is exactly the case where AP and NDCG disagree. RR is 1 for both queries: the first result is always relevant.

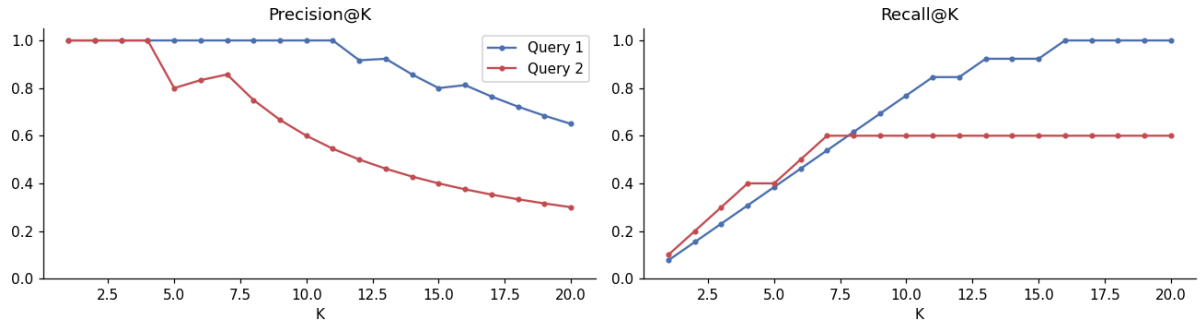


Figure 2. Precision@K and Recall@K for K = 1..20 on the validation queries.

Test queries. The mean values are very high (MAP@10 = 0.981, MRR = 1, NDCG@10 = 0.981), but they must be read with care:

- Queries 2 and 3 have 20 relevant products out of 20 judged, and query 1 has 19. P@K and AP@K are perfect, but the metrics cannot tell whether the ranking would stay good after rank 20.
- R@20 = 1 for every query **by construction**: R only counts relevant products inside the judged top 20, so recall here says nothing about products the system missed. The validation set, judged independently of our system, is the one that revealed missing products (query 2).
- The errors appear in the narrow queries. Query 4 has only 37 AND matches and query 5 only 27: the good products are found first, and then the list is filled with products that contain all the query terms only in the description or the details (loafers and party-wear shoes for "men running shoes", shirts and T-shirts that mention a denim jacket for query 5). None of them has the product type in the title.
- The only error near the top is the pack of socks at rank 4 of query 4 (P@5 = 0.8, NDCG@5 = 0.854). Its text contains "men", "running" and "shoes", and TF-IDF over a merged field cannot know that "socks" is the product while "running shoes" is only the context.
- The track pants at rank 20 of query 1 are the same problem: they contain polo and tshirt outside the title.

Overall, when the user asks for a common product the TF-IDF ranking puts the relevant products at the top, because the query terms are repeated in the title, the details and the description of the right products. The weak points are recall (strict AND, missing words) and precision at the bottom of small result lists (terms that match outside the title).

2.5 Limitations of the current system and proposed solutions

Problem	Evidence	Proposed solution
Strict AND hurts recall. One missing or differently written word removes a relevant product.	Validation query 2: 4 of 10 relevant products never retrieved (R@20 = 0.6).	Rank with OR (or "at least m of n terms") and let TF-IDF / BM25 decide; or relax the query automatically when the AND result is small (drop the term with the lowest idf).
Vocabulary mismatch. Users and sellers use different words (sneakers / shoes, hoodie / hooded, skinny / slim, navy / blue).	Q3 only works because "Hooded: Yes" is in the details; other relevant products may use other words.	Extend the Part 1 normalisation dictionary with synonyms; query expansion with a thesaurus or with word embeddings trained on the corpus; colour families (navy → blue).
All fields are merged. A term in the description counts as much as one in the title.	Socks (Q4, rank 4), track pants (Q1), shirts and vests (Q5) match through their descriptions.	Fielded index with field weights (BM25F, title > details > description), or require the product-type term to be in the title or sub_category.
No phrases or term proximity. "running shoes" and "denim jacket" are matched as independent words.	Products that say "wear it with your running shoes" or "pair it with a denim jacket" are retrieved.	Store term positions in the postings (positional index) and support phrase queries or give a bonus to adjacent query terms.
Ranking quality of plain TF-IDF. No saturation of tf and many exact ties between near-identical products.	Several results with identical scores (e.g. 2.5861 three times for validation query 1); 87 % of the products share their title with another one.	BM25 (tf saturation and tuned length normalisation); break ties with quality signals (rating, stock) and diversify the top results.
Structured fields are not used. Gender, colour, fabric and price are attributes, not just words.	A product described as "for men and women" matches both genders; queries like "shirts under 500" are impossible.	Detect attributes in the query and use the product_details values (Ideal For, Color, Fabric) and the numeric fields of Part 1 as filters or ranking features.
Stemming errors. Porter merges words with different meanings.	"running" → run matches any text about running; "sleeves" → sleev.	Use a lemmatiser or a domain exception list for important product words.
Query processing cost. Every query converts the full postings lists to Python sets.	Lists of up to 20,784 products for words such as men, women or cotton.	Keep postings sorted and intersect starting from the shortest list; stop words specific to the domain (wear, western) could be skipped in the filter.
Small and biased evaluation. Only 20 products per query, binary labels, judged on our own system's output.	R@20 = 1 for all test queries by construction; RR = 1 everywhere, so MRR does not separate systems.	Judge deeper pools or pool the results of several systems; graded labels for NDCG; more queries and inter-annotator agreement (Cohen's kappa).

Table 9. Main problems found and how we plan to solve them.

Use of AI tools

We used Claude (Anthropic) during this part of the project. It helped us write the first version of the indexing, ranking and evaluation functions (`myapp/search/algorithms.py` and `myapp/search/evaluation.py`) and the structure of the notebook, helped us debug specific problems and helped us draft and structure this report. We reviewed, ran and modified all the code, and checked every number in this report against our own notebook outputs. The choice of the test queries, the relevance labels and the interpretation of the results are our own work.